



KINJAL

Kinjal: Views and Validation

Campus Placement and Training Cell. Everything a person sees, types or uploads. Every line number below was checked against the current source.

1. Your role in one sentence

You own the last step of every request: the JSP pages that turn data into HTML, the rules that decide whether typed input is acceptable, and the two ways a file gets in (the resume upload and the bulk import).

2. What you own

FILE	LINES	WHAT IT IS
layout-top.jspf	146	Shared page opening: theme cookie, header, navigation, flash messages.
layout-bottom.jspf	16	Shared footer, with the one <code>jsp:useBean</code> and one <code>fmt</code> tag.
icons.jspf	48	Hidden SVG sprite holding every icon.
Student pages: <code>student-dashboard</code> 191, <code>drives</code> 106, <code>drive-detail</code> 150, <code>my-applications</code> 76, <code>profile</code> 186	709	What a student sees. <code>profile.jsp</code> holds the details form, password form and resume upload.
Officer pages: <code>admin-dashboard</code> 106, <code>admin-students</code> 88, <code>admin-student-detail</code> 140, <code>admin-companies</code> 131, <code>admin-drives</code> 224, <code>admin-applications</code> 107, <code>admin-shortlist</code> 133, <code>admin-audit</code> 70, <code>admin-system</code> 76	1075	What the placement officer sees.
admin-import.jsp	81	The bulk import page (a text box and one button).
<code>error-403</code> 34, <code>error-404</code> 30, <code>error-500</code> 40	104	Error pages wired in <code>web.xml</code> .
<code>assets/app.css</code> 510, <code>app2.css</code> 392	902	All styling, light and dark palettes as colour tokens.

<code>assets/app.js</code>	136	The only script in the app: confirm dialogs and the import button.
<code>assets/favicon.svg</code> 11, <code>assets/fonts/</code> (four Plus Jakarta Sans woff2 files)	11	Icon and self-hosted fonts, so nothing loads from another site.
<code>util/Validators.java</code>	118	Server side input rules (email, roll number, phone, name, password, safe number parsing).
<code>util/Branches.java</code>	36	The fixed list of six branches.
<code>util/UploadStore.java</code>	138	Where resumes live, safe names, file type checks.
<code>web/ProfileServlet.java</code>	236	Profile page: details, password change, resume upload.
<code>web/ResumeDownloadServlet.java</code>	80	Serves a stored resume back, with an owner check.
<code>web/BulkImportServlet.java</code>	218	Non blocking bulk import of students.
<code>ValidatorsTest</code> 153, <code>BranchesTest</code> 50, <code>UploadStoreTest</code> 93	296	JUnit 5 tests for your three util classes.

Not yours: `login.jsp`, `register.jsp`, `admin-reports.jsp`, `admin-database.jsp`, `admin-classroom.jsp`. You will still be asked about the register page, so know it. Also note `webapp/index.jsp` (18 lines): the welcome file, the only JSP outside WEB-INF, no markup in it.

3. The 60 second explanation

SAY THIS

My part is everything the user touches. Pages are JSP with JSTL and EL only. Scriptlets are switched off in web.xml, so no Java can be written into a page. A servlet puts its data on the request and forwards to a JSP under WEB-INF, which is why no page can be opened by URL. Every page includes the same header and footer with the include directive, and user supplied text is printed with `c:out` so it is escaped.

Input is checked twice. The forms carry HTML attributes as hints, but the server decides, using the Validators class, and shows the error next to the field. All JavaScript is in one file, app.js, because our security policy forbids inline scripts. I also own two file paths in: the resume upload, limited by size, extension and the first bytes of the file, stored under a generated name and served back through a servlet; and the bulk import, which reads the request without holding a thread, using a ReadListener.

4. Code walk in VS Code

- 1 **WEB-INF/web.xml , lines 50 to 70.** Point at the two `jsp-property-group` blocks and the line `<scripting-invalid>true</scripting-invalid>` (lines 58 and 67, for `*.jsp` and `*.jspx`). Say: "Scriptlets are refused by the container. A page with Java in it would not even compile, so every page has to use EL and JSTL."
- 2 **web/Web.java , lines 32 and 44 to 49.** Point at `VIEW_ROOT = "/WEB-INF/views/"` and `getRequestDispatcher( ... ).forward( ... )`. Say: "Every servlet ends with this call. The JSPs sit under WEB-INF, so the container will not serve them directly. Only a forward from inside the server can reach them."
- 3 **DriveBrowseServlet.java , lines 55, 65 and 70, then drives.jsp .** Point at `request.setAttribute("drives", drives)`, then `<c:forEach var="d" items="${drives}">` at line 33 of the JSP. Say: "The servlet puts the list on the request. The JSP reads it back by name with EL. That is the whole handover."
- 4 **layout-top.jspf , lines 11 to 13, 25 and 29.** Point at the three `c:set` lines (`ctx` , `me` , `theme`), the `cookie.portal_theme` expression, and `<html lang="en" data-theme="${theme}">`. Say: "The theme comes out of a cookie through the `cookie` implicit object. The page title is printed with `c:out`."
- 5 **drives.jsp line 4 and last line, layout-bottom.jspf line 6.** Point at `<%@ include file="layout-top.jspf" %>`, then `jsp:useBean` and the `fmt:formatDate` on line 12. Say: "The include directive pastes the fragment in at translation time, so the variables it sets are visible to the page. `jsp:useBean` is the one standard action we use."
- 6 **layout-top.jspf , lines 133 to 145.** Point at the flash message block and `<c:remove var="flashSuccess" scope="session"/>`. Say: "A message is shown once and removed, so a refresh does not repeat it."
- 7 **drives.jsp , lines 38 to 40 and 73 to 97.** Point at `<c:out value="${d.company.name}"/>` and the `c:choose` with `c:when` and `c:otherwise` for Already applied, Eligible, Not eligible. Say: "Text comes from the database, so it is always escaped. The verdict was decided in the EJB, the page only shows it."
- 8 **profile.jsp lines 19 to 40, ProfileServlet.java lines 91 to 121.** Point at `novalidate` , `required` , `min` , `max` , `aria-invalid` , the `div class="err"` ; then the `errors` map in the servlet. Say: "The attributes are hints. The servlet runs Validators, collects messages in a map, and forwards back to the same page, where each message sits under its field."
- 9 **util/Validators.java lines 6 to 10, 20 to 29, 61.** Point at the class comment, the `EMAIL` pattern, and `isStrongPassword`. Say: "Deliberately stricter than the RFC. `admin@campus` is refused. The HTML attributes are not a security control, this class is." Then run `.\test.ps1` if asked: 27 test methods across your three test classes.

10 `assets/app.js` lines 1 to 31, `security/SecurityHeadersFilter.java` lines 51 to 61. Point at the header comment and the `submit` listener that reads `data-confirm`, then `script-src 'self'` in the CSP. Say: "The browser runs only scripts served from our own origin. So no page may contain a script block or an onclick. The confirm dialogs use a data attribute and one listener."

11 **Upload path.** `profile.jsp` lines 154 to 165 (`enctype="multipart/form-data"` , `accept`), `ProfileServlet.java` lines 32 to 35 (`@MultipartConfig` , 2 MB file, 3 MB request), 73 to 82 (an oversize file is caught here) and 159 to 214, `UploadStore.java` lines 36 to 39 and 92 to 107, and `security/AuthFilter.java` lines 70 to 80 (the same oversize catch, because the filter reads the token first). Say: "Size first, then extension, then the opening bytes must match the format, then the file is written under a name we generate."

12 `ResumeDownloadServlet.java` lines 41 to 78. Point at the owner check, `UploadStore.resolve` , `Content-Disposition: attachment` , `nosniff` . Say: "The file is found from the student record, never from a name in the URL. A student gets 403 for anyone else's resume."

13 **Bulk import.** `BulkImportServlet.java` line 40 (`asyncSupported = true`), line 67 (`startAsync`), lines 73 to 118 (`ReadListener` : `onDataAvailable` , `onAllDataRead` , `onError`); `admin-import.jsp` lines 36 to 39; `app.js` lines 109 to 135. Say: "The button posts the pasted text with fetch. The servlet reads it in chunks only when the container says bytes are ready."

14 `assets/app.css` line 102 and `web/ThemeServlet.java` . Point at `[data-theme="dark"]` replacing the colour tokens. Say: "Dark mode is a token swap, not a second stylesheet."

5. Live demo plan

Prepare before the demo: one small real PDF (under 2 MB) on the desktop, one text file renamed to `fake.pdf` , and one file over 2 MB (any large file, for example a video or a big zip). When this was written nobody had a resume on file, so the upload has to come first.

1 **Direct JSP is refused.** Open `http://localhost:8080/placement/WEB-INF/views/drives.jsp` . You get the "There is nothing at that address" page (a 404, not 403). Say: "The container hides WEB-INF. The only way to that file is a forward."

2 **Sign in as the student.** `http://localhost:8080/placement/login` , `aarti.deshpande@campus.edu` , `Campus@2026` . You land on `/app/dashboard` . Click the moon icon at top right: the address flashes `/theme?mode=dark&back= ...` and returns dark. Click the sun to go back. Say: "A cookie called `portal_theme`, read with EL, sets `data-theme` on the html tag."

- 3 A JSP showing database data.** Click **Drives** (`/placement/app/drives`). You see 10 cards, first **Saffron Retail Tech**. Right click, View page source: one header, escaped text, no script tags except the app.js link. Then say the path: browser, `/app/drives` , `AuthFilter` , `DriveBrowseServlet.doGet` , `DriveService.listOpen()` (a stateless EJB running the `Drive.open` named query), `request.setAttribute("drives")` , `Web.render` , forward to `/WEB-INF/views/drives.jsp` , `c:forEach` , `c:out` . Optional proof: run `select c.name from drive d join company c on c.id=d.company_id where d.status='OPEN' order by d.last_date` in MySQL and the order matches.
- 4 A validation error.** Click **Profile** (`/app/profile`). Set CGPA to `99` and Mobile number to `12` , click **Save details**. The page returns with red text under each field: "CGPA must be between 0 and 10." and "Enter a ten digit mobile number." Nothing is saved, and the boxes show the saved values again. Say: "The browser did not stop me, the form has novalidate. The server decided."
- 5 Escaping.** Open a private window on `http://localhost:8080/placement/register` . Full name `"><b>hi</b></code> , email admin@campus , leave the rest bad, click Create account. The text comes back inside the name box exactly as typed, not as bold. View source shows "><b> . No account is created. Say: "This is c:out."`
- 6 Resume upload and download.** Back on the student **Profile**, right hand card. Choose `fake.pdf` , click **Upload**: red banner "That file does not look like a .pdf document inside, whatever it is named." Now choose the real PDF, click **Upload**: green banner "Resume uploaded." and the card says a resume is on file. Click **Download the copy on file**: the browser saves `Resume-CS21001.pdf` . Say where it lives: `domains/domain1/placement-resumes/resume-1-xxxxxxx.pdf` , outside the deployed app. This changes data: Aarti now has a resume on file.
- 7 Oversize file.** On the student **Profile** choose the file over 2 MB and click **Upload**. You return to the profile with the red banner "That file is larger than the two megabyte limit." Nothing is stored. Say: "The container refuses it, and we turn that into a normal message instead of an error page." For the real upload keep using the small PDF.
- 8 Another student's resume is refused.** Type `http://localhost:8080/placement/app/resume?student=2` while signed in as Aarti. You get the 403 page. Student 2 has no resume, so that is the refusal to show. As the officer, **Students**, Aarti Deshpande, the Resume row shows **Download** (`?student=1`) and it works.
- 9 Bulk import as officer.** Sign out, sign in as `tpo@campus.edu` , `Campus@2026` . Open **Tools**, then **Import** (`/placement/admin/import`). Click **Import these rows** with the pre filled text. Today's result: 1 created (`IT22011` , Karan Bhatt), 2 refused: row 1 "An account already exists for priya.nair@campus.edu" (she is already seeded) and row 3 "broken-address is not a valid email address". Click **Open the student register** to show Karan Bhatt. This creates a real account: remove it afterwards from **Students**, open Karan Bhatt, **Remove from the register** (that button uses the data-confirm dialog from app.js).

- 10 Role guard on a page.** Optional, as the student: `http://localhost:8080/placement/admin/import` shows the 403 page rendered from `error-403.jsp`.

CAREFUL

This is a different, separate fake student from the "Demo Student" the rest of the presentation uses, so it does not collide with that account. Rehearse the import with the first two rows changed to a new fake name, email and roll number (for example `Priya Patil,priya.patil.import@campus.edu,CS22099,Computer Science,2027,7.5,0,9820100299`) so the real demo still has something to create. A second run of the same rows creates nothing.

6. Servlet and JSP course topics mapped to this code

TOPIC	WHERE IN THE CODE
Page directive	Line 1 of every view: <code><%@ page contentType="text/html; charset=UTF-8" %></code> . <code>error-500.jsp</code> adds <code>isErrorPage="true"</code> .
Taglib directive	<code><%@ taglib prefix="c" uri="jakarta.tags.core" %></code> on every page, plus <code>fmt</code> in <code>layout-top.jspf</code> .
Include directive	Every page: <code><%@ include file="layout-top.jspf" %></code> first, <code>layout-bottom.jspf</code> last. <code>layout-top.jspf</code> includes <code>icons.jspf</code> .
Standard action	<code>jsp:useBean</code> in <code>layout-bottom.jspf</code> line 6. This is the only one. <code>jsp:include</code> and <code>jsp:forward</code> are not used.
JSTL core	<code>c:set</code> , <code>c:if</code> , <code>c:choose/when/otherwise</code> , <code>c:forEach</code> , <code>c:out</code> , <code>c:remove</code> in the views; <code>c:redirect</code> in <code>index.jsp</code> .
JSTL formatting	<code>fmt:formatDate</code> in <code>layout-bottom.jspf</code> line 12 (the render time in the footer).
EL implicit objects	<code>pageContext</code> (<code>pageContext.request.contextPath</code> , <code>pageContext.exception</code>), <code>sessionScope</code> (<code>sessionScope.user</code> , flash messages), <code>requestScope</code> (the forward path, <code>layout-top.jspf</code> line 23), <code>cookie</code> (<code>cookie.portal_theme</code>). <code>param</code> and <code>header</code> are not used.
EL operators	<code>empty</code> , <code>not</code> , <code>? :</code> , <code>==</code> , <code>eq</code> , map lookup <code>verdicts[d.id]</code> , method call <code>applications.size()</code> .
Scopes	Page: <code>ctx</code> , <code>me</code> , <code>theme</code> . Request: <code>pageTitle</code> , <code>drives</code> , <code>detailErrors</code> . Session: <code>user</code> , <code>flashSuccess</code> , <code>csrfToken</code> .
Servlet to JSP	<code>request.setAttribute</code> then <code>Web.render</code> , which calls <code>RequestDispatcher.forward</code> (<code>Web.java</code> lines 44 to 49).
No scriptlets	<code>web.xml</code> <code>scripting-invalid</code> , lines 58 and 67.
Error pages	<code>web.xml</code> lines 33 to 48: 404, 403, 500 and <code>java.lang.Throwable</code> mapped to the three JSPs.
Redirect after POST, flash	<code>Web.redirect</code> , <code>Web.flashSuccess</code> ; <code>ProfileServlet</code> ends every save with a redirect.

Cookies	Written by <code>ThemeServlet</code> , read in <code>layout-top.jspf</code> line 13.
File upload	<code>@MultipartConfig</code> , <code>Part</code> , <code>getSubmittedFileName</code> in <code>ProfileServlet</code> .
File download	<code>ResumeDownloadServlet</code> : content type, <code>Content-Disposition</code> , streaming with <code>transferTo</code> .
Async and non blocking I/O	<code>startAsync</code> , <code>ReadListener</code> , <code>AsyncContext.complete</code> in <code>BulkImportServlet</code> .
Input validation	<code>Validators</code> , <code>Branches</code> , used by <code>ProfileServlet</code> and <code>BulkImportServlet</code> (and by register).

7. Questions you will be asked

Q Why can't I open your JSPs directly?

They live under `WEB-INF/views` , and the container never serves anything under WEB-INF to a browser. The only way in is `RequestDispatcher.forward` from a servlet, which is what `Web.render` does. Try it: `/placement/WEB-INF/views/drives.jsp` gives a 404. The one exception is `index.jsp` , the welcome file, and it holds no markup.

Q Why are scriptlets disabled, and where?

In `web.xml` , `<scripting-invalid>true</scripting-invalid>` for `*.jsp` and `*.jspx` . Java inside a page mixes logic into the view, cannot be tested, and is where injection bugs hide. With it off, a page containing a scriptlet fails to compile, so the rule cannot be broken by accident.

Q Include directive or `jsp:include` ? Which do you use?

The include directive, `<%@ include file="layout-top.jspf" %>` . It is a static include: the fragment is pasted in when the page is translated, so the whole page becomes one servlet and variables like `ctx` and `me` set in the fragment are visible to the page. `jsp:include` would run at request time as a separate resource. We do not use it.

Q What does `c:out` do, and why is it on every user value?

It escapes `&` `<` `>` `"` `'` before printing. Without it, a name like `"<script>alert(1)</script>` would be written into the page as live markup. With it, the register page sends back `"><script>alert(1)</script>` , which the browser shows as plain text. Plain EL like `${x}` does not escape. I checked this against the running register page.

It is used inside attributes too, not only in page text. The `data-confirm` messages that contain a name (`admin-companies.jsp` line 52, `admin-student-detail.jsp` lines 101 and 129, `my-applications.jsp` line 52) use `<c:out value='${co.name}' />` inside the attribute. Plain EL remains only for numbers and server made values such as ids, the csrf token and status labels. One client supplied exception: the theme cookie value on the html tag, which only the person's own browser can set.

Q How does a servlet hand data to a JSP?

It calls `request.setAttribute("drives", drives)` and then forwards. The JSP reads `${drives}`. EL searches page, request, session, then application scope for the name. The address bar keeps the servlet URL because a forward happens inside the server.

Q Forward or redirect, and when?

Forward to show a page: same request, attributes survive, no second round trip. Redirect after a successful POST so a refresh does not repeat the write. A message that must survive the redirect goes in the session as `flashSuccess` or `flashError`, and `layout-top.jspf` removes it with `c:remove` after printing.

Q Is validation done in the browser or on the server?

Both, but only the server counts. Inputs carry `required`, `min`, `max`, `maxlength`, `type="email"`, and the forms also carry `novalidate`, so the browser does not block a submit or show its own bubbles. `Validators` does the real check and the page shows the message under the field with `aria-invalid="true"`. The register page says it plainly: "Every field is checked again on the server before anything is saved." Anyone can bypass a browser with curl.

Q Why is there no inline JavaScript?

The Content-Security-Policy header says `script-src 'self'` with no `'unsafe-inline'`, so the browser runs only scripts served from our own origin. An injected script tag would sit there unused. That only works if no page has a script block or an onclick, so everything is in `app.js`. Confirm dialogs use a `data-confirm` attribute and one listener. Inline *styles* are allowed by the policy and a few pages use them.

Q How does light and dark mode work?

Clicking the icon calls `/theme?mode=dark`. `ThemeServlet` stores `portal_theme` in a one year cookie, accepting only `light` or `dark`. Each page reads it with `cookie.portal_theme.value` and puts it on the html tag as `data-theme`. `app.css` line 102 swaps the colour tokens under `[data-theme="dark"]`. It does not follow the operating system setting.

Q How is the resume upload kept safe?

Four checks and one placement rule (plus a friendly message for an oversize file). `@MultipartConfig` caps a file at 2 MB and a request at 3 MB. The extension must be pdf, doc or docx. The first bytes must match that format (`%PDF`, or the zip or OLE signature). An oversize file is caught and answered with a normal message (`ProfileServlet` lines 73 to 82, `AuthFilter` lines 70 to 80). The saved name is generated (`resume-1-a1b2c3d4.pdf`), so the browser's file name is never used as a path. The file goes to `domain1/placement-resumes`, outside the deployed app, so a redeploy does not wipe it.

Q How is a resume served back, and can a student read someone else's?

Through `/app/resume`, not a static link. The servlet finds the file name from the student record, resolves it inside the upload folder only, and sends it with `Content-Disposition: attachment` and `nosniff`. A student asking for another student's id gets 403. The officer may fetch any.

Q Explain the non blocking bulk import in plain words.

Normally `read()` makes a server thread wait for the next byte, so a slow upload ties up a thread doing nothing. In `BulkImportServlet` the request is put in async mode (line 67) and a `ReadListener` is attached (line 73). The container calls `onDataAvailable` only when bytes are ready, we read while `isReady()` is true, then the thread goes back to the pool. When the body ends, `onAllDataRead` validates the rows, writes JSON and calls `async.complete()`. Limits: 512 KB and a 30 second timeout. The non blocking part is reading the request. The database work for each row still runs on that callback thread.

Q Is the import a file upload?

No. The officer pastes text into a box and `app.js` sends it with `fetch` as `text/csv`. It is not multipart, which is why it is a different mechanism from the resume upload. The CSRF token goes in the query string because there is no form body to carry it. The script writes the JSON reply into the page with `textContent`, never `innerHTML`.

Q What accessibility work did you do?

What is in the markup: `lang="en"` on every page; a `label for` on form fields; `aria-invalid` set by the server with visible error text, so colour is not the only signal; `aria-hidden` on the logo and icon sprite; `aria-label="Menu"` on the menu toggle; a visible `:focus-visible` outline, including on the CSS only menus; transitions only when the user has not asked for reduced motion; and a written contrast audit in the comments of `app.css`. Not done: no skip link, and the theme and sign out icons have `title` text but no `aria-label`.

8. If something goes wrong

WHAT YOU SEE	WHY, AND WHAT TO DO
Red banner "That file is larger than the two megabyte limit."	The file is over 2 MB. That is the intended message. Use the small PDF for the real upload.
Officer opens <code>/app/profile</code> and lands on the dashboard	Expected. An officer has no profile page, so <code>ProfileServlet</code> redirects to <code>/admin/dashboard</code> . Show the upload as the student.
Upload refused	The name must end in <code>.pdf</code> , <code>.doc</code> or <code>.docx</code> and the content must match. A renamed text file is refused on purpose.
Import says "The import could not be completed"	Session expired, so the request was bounced to the login page. Sign in again, reopen Import.

Import creates nothing	Those emails or roll numbers already exist, for example a second run. Change the email and roll number in the rows.
Download says no resume	Nothing has been uploaded for that student yet (nobody has one at present). Upload first.
Theme does not switch	Cookies blocked in this browser window. Use a normal window, then hard refresh.
Page unstyled or icons missing	Hard refresh (Ctrl+F5). Check <code>/placement/assets/app.css</code> returns 200.
Every page fails, connection refused	The server is down. Tell the team member who runs it. Do not start it yourself mid demo.